

Qubit Level Operation on Quantum Testbed

Faculty of Environment and Information Studies

Academic Year 2026

B4

Shen Yifeng

tomshen

July, 2026

Abstract

A quantum network contains many quantum nodes, such as Bell state analyzers (BSA), optical switches (OSW), and entangled proton pair sources (EPPS). As these networks increase in size and complexity, the management of these nodes will become increasingly challenging. Existing nodes are device-centric, meaning that they have to be controlled at the device level, requiring operators to interact directly at the device level, and writing programs to control the device directly. Furthermore, there are different variations in the design of these nodes; therefore, combining them into a network can be very difficult.

This thesis proposes a logical node abstraction that decouples the network control functions from the physical hardware devices. The proposed control software provides high level networking operations interface, and translates these to device instructions underneath the hood. This abstraction enables applications to control quantum network nodes without requiring detailed knowledge of the device-specific implementation as long as the physical design matches a supported configuration.

As part of this thesis, a software called `logical_node_service` was created which contains the proposed capabilities. The ability to perform state tomography on a single-qubit MEAS node was implemented to demonstrate the usability of the software. This software was tested on the Moonshot Nagayama Project networking testbed to verify the functionality of the MEAS node. Our software showed that logical abstraction can greatly simplify the process of managing these nodes compared to the existing solution on the testbed.

Contents

1	Introduction	1
1.1	Research Question	2
1.2	Research Contributions	2
1.3	Thesis Structure	3
2	Background	4
2.1	Quantum Information	4
2.1.1	Qubits and quantum states	4
2.1.2	Entanglement and Bell states	6
2.1.3	Quantum gates	6
2.1.4	Measurement and basis	8
2.2	Quantum Communication	9
2.2.1	Introduction to quantum networks	9
2.2.2	No cloning theorem	10
2.2.3	Quantum teleportation	10
2.2.4	Entanglement swapping	12
2.2.5	Network Architectural Planes	14

2.2.6	Architecture of quantum network	15
2.2.7	Quantum data center network (QDCN) and Q-Fly	16
3	Quantum Network Testbed Nodes	18
3.1	Testbed	18
3.2	MEAS node	19
3.2.1	Quantum state tomography	20
3.2.2	Functionality of logical MEAS node	21
3.3	Other logical nodes	22
3.3.1	EPPS node	22
3.3.2	OSW node	23
3.3.3	BSA node	23
4	Testbed Hardware	25
4.1	Thorlabs APT Protocol	25
4.2	Motorized Cage Rotator	26
4.3	Optical Delay Line	26
4.4	Motorized Polarization Controller	27
4.5	Optical Switch	29
4.6	SNSPD and TDC	29
5	Testbed Software Architecture	30
5.1	General Architecture	31
5.2	Driver Packages	31
5.2.1	pnpq	31

5.2.2	<code>tdc_toolkit</code>	33
5.3	Service-oriented architecture	33
5.3.1	Benefits of microservices	34
5.3.2	Schema First Development	34
5.4	gRPC Services Packages	35
5.4.1	<code>pnpq_service</code>	36
5.4.2	<code>tdc_service</code>	37
6	Testbed Logical Node	38
6.1	Design Overview	38
6.1.1	Physical device manager	38
6.1.2	Logical node manager	39
6.1.3	Experiment manager	39
6.1.4	Documentation	40
6.2	Logical Node Designs	40
6.3	Implementation Details	41
7	Evaluation	42
7.1	Functionality Check	42
7.2	Quality Analysis	43
7.2.1	Usability and simplicity	44
7.2.2	Reusability	45
7.2.3	Extensibility	46
7.2.4	Portability	47

7.3	Overhead	47
7.4	gRPC service based system and event broker system	48
8	Conclusion	49
8.1	Contributions	49
8.2	Future works	50
8.2.1	Implement other nodes	50
8.2.2	Open source project and spread adoption	50
8.2.3	Support higher level protocol	50

List of Figures

2.1	Bloch sphere showing state $ \psi\rangle$ in relation to equation 2.3.	5
2.2	Quantum circuit for quantum teleportation	12
2.3	Before measuring X_2 and Y_1 at Bob's node	12
2.4	After measuring X_2 and Y_1 at Bob's node and sharing information	12
2.5	Entanglement swapping of 2 Bell pairs.	14
2.6	Simple quantum network with 2 nodes and a repeater.	16
2.7	Types of Q-Fly networks. (Diagram from [25])	17
3.1	A single group within a Q-Fly network, consisting of three quantum end nodes and a switching BSA node. (Diagram from [25])	19
3.2	A single measurement node.	20
3.3	Image of a MEAS node connected to one output of the EPPS on the testbed.	20
3.4	Two quantum end-nodes connected through a Bell State Analyzer (BSA).	24
4.1	Image of Thorlabs K10CR2. (Image from [33])	27
4.2	Image of Thorlabs K10CR2 with Thorlabs KBD101 motor controller. (Image from [32])	28
4.3	Image of Thorlabs MPC320 with 3 channels. (Image from [34])	28
5.1	General architecture of the testbed software.	30

5.2	Communications between <code>pnpq_service</code> and different types of clients	36
6.1	Schema design of logical nodes.	41
7.1	Service architecture tree for <code>pnpq_service</code> and <code>tdc_service</code>	44
7.2	Comparison of two possible hardware configurations of one MEAS nodes on two different testbeds.	46

List of Tables

2.1	Bob's state after Alice's Bell measurement and the required correction. . . .	11
2.2	Bell state that Alice and Carol hold after Bob performs BSM.	14
3.1	Angles for waveplates for measuring various bases using a MEAS node. (Table adapted based on information from [12])	20
4.1	Supported Commands for Thorlabs APT Devices	26
7.1	Detector count returned from the MEAS node for each bases.	43
7.2	Evaluation Criteria	44

Chapter 1

Introduction

The main function of a quantum network is to achieve quantum entanglement distribution [38]. By doing this, applications such as distributed quantum computing, quantum communication, and quantum key distribution are made possible [5, 15, 7, 20].

Routing quantum traffic in a memory-less network can be achieved using a link architecture using several quantum nodes such as Bell State Analyzers (BSAs), the Entangled Proton Pair Source (EPPS), and optical switches [13, 27]. Routing protocols are also being developed to make efficient use of these network node resources to route quantum network traffic between nodes [30].

In classical networking, there were many attempts to solve the problem of incompatible hardware designs between different vendors or even between different models within the same vendor.

One example is Software Defined Networking (SDN). SDN has become popular among many network administrators and cloud service providers by introducing a control layer that decouples from the specific implementation of the hardware, allowing it to scale easily [18]. Similarly, a quantum device can also benefit from this abstraction because it can enable heterogeneous hardware implementation to work together [21].

OpenConfig is another platform that aims to provide vendor-independent network management by using a common model. It provides an interface for both the configuration and the control network hardware [23]. Using this model, it is also possible to develop microservices that provide automation capabilities on networking hardware [2].

With the development of protocols that nodes can implement to achieve the goal of a quantum network and the realization of several designs of the network nodes, it is now important to have a control system that can operate on these nodes. At the moment, control systems for testbed nodes, including the one in the Moonshot Nagayama Project quantum testbed, are device-centric. Before each node can be operated, full knowledge of the underlying hardware must be obtained. Therefore, it becomes necessary to examine the level of abstraction that will simplify the management of network nodes within the testbed.

1.1 Research Question

In this thesis, the question of how logical abstraction can improve the ability to control and manage quantum networks with multiple nodes will be examined.

A control software called `logical_node_service` will be created as part of the process. This library implements the several techniques that will be discussed in this thesis. It will support the devices on the quantum network testbed within Moonshot Nagayama Project's laboratory.

1.2 Research Contributions

This thesis presents several contributions, shown below.

1. **Defined logical node units** - Although the specifications of many quantum nodes are available, they are generally abstract, do not contain concrete specifications, and do not define concretely how it will be operated. In this thesis, these will be defined using the setup on the quantum network testbed.
2. **New control architecture** - A new control architecture is designed to go beyond the physical devices. This architecture provides a higher-level logical control interface that operates on a quantum node layer rather than at the physical hardware level. It is also expandable, which means that other laboratories can adapt the same control interface to support their labs.
3. **The implementation** - `logical_node_service` will be created with the specifications defined in this thesis.

4. **Analysis of effectiveness** - The product will first be evaluated on whether it meets the functional requirements. Then, non-functional software quality analysis will be used to identify strengths and weaknesses of the approach based on several defined criteria.

1.3 Thesis Structure

This thesis will start with a discussion of the background concepts in quantum information processing as well as the basic concepts of a quantum network in Chapter 2. Then, Chapter 3 will cover the nodes available in the hardware and their designs which will be implemented within the proposed software. Chapter 4 covers some of the hardware that is necessary to support as part of the underlying infrastructure and how they contribute to the quantum network nodes. Chapter 5 covers the process of designing and building dependencies software and library frameworks for the proposed software. Finally, Chapter 6 will introduce the software itself and analysis of effectiveness will be done in Chapter 7.

Chapter 2

Background

This chapter explains the basic prerequisite knowledge required to understand the problem and solution which will be introduced in the subsequent chapters. Many quantum elements will be introduced by comparing them with their classical counterparts and bringing in additional capabilities unique to the quantum world.

2.1 Quantum Information

Unlike classical computers, which use classical bits, quantum computers use quantum bits known as qubits.

2.1.1 Qubits and quantum states

Qubit is the fundamental processing unit of a quantum computer [22].

A classical bit can have two possible states: 0 and 1. This can be compared to a coin placed on the table. It can be either head or tail. Similarly, a qubit can also be in the basis states $|0\rangle$ and $|1\rangle$. However, unlike the classical bit, a qubit can also be in a *superposition* of these two states until it is measured.

A qubit can be described using the following equation:

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle, \quad (\alpha, \beta) \in \mathbb{C}, \quad |\alpha|^2 + |\beta|^2 = 1 \quad (2.1)$$

The two basis states ($|0\rangle$ and $|1\rangle$) are formed by setting one of α or β to 1, and the other to 0. What makes a qubit special is that as long as α and β satisfy the conditions of the qubit equation, they are not limited to only 0 and 1. This means that the state is somewhere between 0 and 1, which puts the qubit in a superposition.

An example of a frequently used set of quantum states in superposition is the $|+\rangle$ and $|-\rangle$ states. These states are in the following form:

$$|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle), \quad |-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) \quad (2.2)$$

The qubit state equation can also be rewritten in terms of the functions $\cos$ and $\sin$, as shown in equation 2.3. Here, θ is the polar angle and φ is the phase angle. Together, they specify a point on the Bloch sphere, shown in Figure 2.1, which represents the qubit state $|\psi\rangle$.

$$|\psi\rangle = \cos \frac{\theta}{2} |0\rangle + e^{i\varphi} \sin \frac{\theta}{2} |1\rangle, \quad \theta \in [0, \pi], \quad \varphi \in [0, 2\pi) \quad (2.3)$$

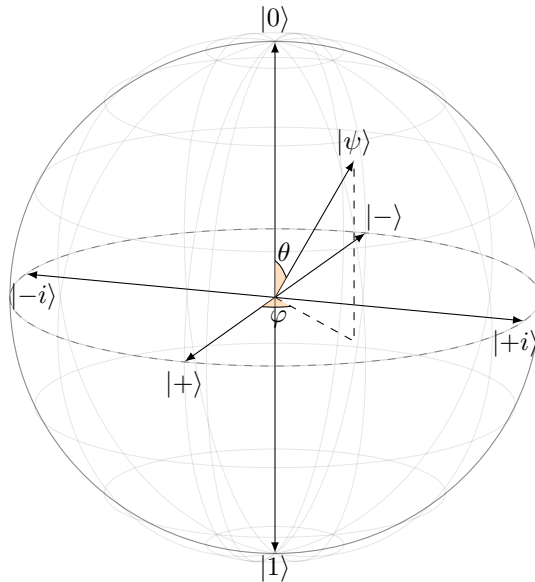


Figure 2.1: Bloch sphere showing state $|\psi\rangle$ in relation to equation 2.3.

2.1.2 Entanglement and Bell states

Having only one qubit does not enable very useful quantum computing capabilities. Therefore, multiple qubits need to work together in order for quantum algorithms to work.

Multiple classical bits can be placed together to make different permutations. For example, two bits can have the following four possible patterns: 00, 01, 10, 11. Qubits can also have the four possible *computational basis states*: $|00\rangle$, $|01\rangle$, $|10\rangle$, $|11\rangle$.

A multiple qubit state can be described using a tensor product of the individual qubit states, as shown in equation 2.4. These are called *product states*.

$$\begin{aligned} |\phi\rangle_A &= |0\rangle, \quad |\phi\rangle_B = |1\rangle \\ |\phi\rangle_{AB} &= |\phi\rangle_A \otimes |\phi\rangle_B = |01\rangle \end{aligned} \tag{2.4}$$

In addition to product states, there are also multiple qubit states that cannot be composed of two individual states using tensor products. These states are *entangled*.

The most commonly used sets of entangled states are called *Bell states*. Bell states are maximally entangled states. There are four of them.

$$\begin{aligned} |\Phi^+\rangle &= \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle), \quad |\Phi^-\rangle = \frac{1}{\sqrt{2}}(|00\rangle - |11\rangle) \\ |\Psi^+\rangle &= \frac{1}{\sqrt{2}}(|01\rangle + |10\rangle), \quad |\Psi^-\rangle = \frac{1}{\sqrt{2}}(|01\rangle - |10\rangle) \end{aligned} \tag{2.5}$$

Entangled qubits share a common qubit state, whereas a product state does not. It is not possible to express this pair as a product state, meaning that it is not possible to describe only one of the two qubits in an entangled pair.

2.1.3 Quantum gates

A classical computer uses logic gates such as AND or OR in order to manipulate one or more classical bits. Similarly, a quantum computer uses its own sets of quantum gates to perform

operations on qubits.

The only criterion for a quantum gate is that the gate must be *unitary*. This means that a gate has to be represented using a unitary matrix. A matrix U is unitary when $U^\dagger U = I$, where $U^\dagger$ is the *adjoint* (the complex conjugate transpose matrix) of U .

Pauli gates

The simplest gate available on a classical computer is the NOT gate. This gate changes a 0 to a 1, and vice versa. On the other hand, quantum computers have the Pauli gate. The Pauli gate operates on a single qubit and performs a transformation on an axis of the Bloch sphere.

There are three types of Pauli gates, X , Y , and Z . Each gate rotates the qubit states on the respective axis on the Bloch sphere.

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \quad (2.6)$$

The Pauli X gate is the quantum counterpart of the classical NOT gate - it converts the state $\alpha|0\rangle + \beta|1\rangle$ to $\beta|0\rangle + \alpha|1\rangle$, essentially flipping the bit.

The Pauli Z gate does not do anything to the state $|0\rangle$, but flips $|1\rangle$ to $-|1\rangle$, introducing a phase change.

The Pauli Y gate performs both a bit flip and a phase change.

Hadamard gate

The Hadamard gate is said to be one of the most useful quantum gate.

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \quad (2.7)$$

This gate first rotates the qubit $\frac{\pi}{2}$ along the y-axis of the Bloch sphere, then π along the x-axis. This gate can be used to create a superposition from one of the basis states. Also, since $H^2 = I$, applying this gate twice returns the qubit to its original state.

CNOT gate

A CNOT (Controlled NOT) gate operates on two qubits, a control qubit and a target qubit. This gate applies the Pauli X operation to the target qubit only if the control qubit is in the 1 state. This gate creates a dependency between the two qubits, therefore creating an *entanglement*.

$$\text{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} \quad (2.8)$$

For the computational basis states, the CNOT gate does the following transformation:

$$\text{CNOT}(|00\rangle) = |00\rangle, \text{CNOT}(|01\rangle) = |01\rangle, \text{CNOT}(|10\rangle) = |11\rangle, \text{CNOT}(|11\rangle) = |10\rangle \quad (2.9)$$

CNOT gate can also create entangled qubit pairs by operating on a qubit that is in superposition:

$$(H \otimes I) |00\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |10\rangle), \quad (2.10)$$

$$\text{CNOT} \left(\frac{1}{\sqrt{2}}(|00\rangle + |10\rangle) \right) = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle), \quad (2.11)$$

In the above example, Equation 2.10 first converts the pair into a product state $|+\rangle \otimes |0\rangle$. Then, in Equation 2.11, using the CNOT gate, the 2 qubits will become entangled because the second qubit now has a dependency on the first qubit.

2.1.4 Measurement and basis

A classical bit can be examined to retrieve its value and find out whether it is 0 or 1. However, it is not possible to get the original qubit state back in the form of equation 2.1 by examining a qubit. Rather, only limited information can be retrieved by using a process called *measurement*.

When measuring a single qubit state on z-basis, it can return either 0 or 1 because it being in superposition. The probability of returning 0 is $|\alpha|^2$, while the chance of returning 1 is $|\beta|^2$. Also, measuring a qubit collapses it to the state returned by the measurement. For example, if the result of the measurement for a qubit is 1, then the qubit becomes the state $|1\rangle$.

The Z-basis is only one of the many bases to measure a qubit. For example, it is also possible to measure in the X-basis. If the same qubit state is rewritten in terms of the $|+\rangle$ and $|-\rangle$ state (shown in equation 2.12), the measurement will result in $+$ with probability $(\alpha + \beta)/2$ and $-$ with probability $(\alpha - \beta)/2$.

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle = \frac{\alpha + \beta}{\sqrt{2}} |+\rangle + \frac{\alpha - \beta}{\sqrt{2}} |-\rangle. \quad (2.12)$$

2.2 Quantum Communication

The quantum network differs significantly from the classical network. The classical network can directly examine a bit packet in order to perform actions such as copying, error correction. However, as described in the previous section, it is impossible to examine a qubit and get the original state. Therefore, many aspects of quantum networking differ greatly from its classical counterpart.

2.2.1 Introduction to quantum networks

The goal of a classical network is to connect computers together in order for them to exchange data between each other. They route network data packets from one computer to another over a network of switches. A quantum network, as its classical counterpart, works to connect quantum computers together [41, 14].

A quantum network is made-up of a network of nodes. There are 4 main types of nodes, each participating in a different role in the network [39]:

1. **End nodes** - Nodes in the network that execute quantum applications. For example, the MEAS node (3.2).

2. **Support nodes** - Nodes in the network that provide a specialized functionality but do not run applications. For example, the EPPS node (3.3.1) and the BSA node (3.3.3)
3. **Repeater nodes** - Intermediate nodes that extend communication distance by performing *quantum teleportation* and error correction.
4. **Composite nodes** - A node that aggregates the functions of multiple nodes.

2.2.2 No cloning theorem

The no cloning theorem is very important and must be kept in mind when designing a quantum network system. Essentially, it states that it is not possible to clone an identical copy of a quantum state that is unknown [42]. This is because to obtain information about the state, it needs to be measured, and measuring the state will cause it to lose its current superposition.

(However, if there is a quantum state that is known, then it can be "cloned" by recreating the qubit from scratch.)

2.2.3 Quantum teleportation

Quantum teleportation refers to the process of sending a piece of quantum information to a receiver some distance away [3]. In order to do this, it requires a classical communication link and a quantum link. Therefore, quantum teleportation cannot be done at a speed faster than the speed of light. One method of performing quantum teleportation is called *entanglement swapping* (Section 2.2.4) [10].

For example, suppose that a sender called Alice wants to teleport this qubit to a receiver called Bob:

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle, |\alpha|^2 + |\beta|^2 = 1 \quad (2.13)$$

First, Alice and Bob need to own a shared bell pair:

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle) \quad (2.14)$$

The state of the 3 qubits (Alice's qubit and the bell pair) is shown using the following tensor product:

$$|\psi\rangle \otimes |\Phi+\rangle = \frac{1}{\sqrt{2}}(\alpha|000\rangle + \alpha|011\rangle + \beta|100\rangle + \beta|111\rangle) \quad (2.15)$$

With this setup, it is possible for Alice to teleport the qubit to Bob by performing a Bell measurement. First, Alice applies the CNOT gate with qubit 0 as the control qubit and qubit 1 as the target qubit. This results in the following state:

$$\frac{1}{\sqrt{2}}(\alpha|000\rangle + \alpha|011\rangle + \beta|110\rangle + \beta|101\rangle) \quad (2.16)$$

Finally, Alice applies the Hadamard gate on qubit 0. Resulting in the following states:

$$\frac{1}{\sqrt{2}}(\alpha|000\rangle + \alpha|100\rangle + \alpha|011\rangle + \alpha|111\rangle + \beta|001\rangle + \beta|101\rangle + \beta|010\rangle + \beta|110\rangle) \quad (2.17)$$

Rearranging the state to factor out Alice's two qubits:

$$\frac{1}{\sqrt{2}}(|00\rangle(\alpha|0\rangle + \beta|1\rangle) + |01\rangle(\alpha|1\rangle + \beta|0\rangle) + |10\rangle(\alpha|0\rangle - \beta|1\rangle) + |11\rangle(\alpha|1\rangle - \beta|0\rangle)) \quad (2.18)$$

At this point, Alice performs the measurement on both qubits, collapsing the qubit state. Then, Alice sends the measurement results to Bob via a classical channel. Finally, Bob can apply the relative correction transformation as according to Table 2.1 to obtain the qubit with which Alice had to start.

m_0m_1	Bob's state	Correction
00	$\alpha 0\rangle + \beta 1\rangle$	I (no corrections)
01	$\alpha 1\rangle + \beta 0\rangle$	X
10	$\alpha 0\rangle - \beta 1\rangle$	Z
11	$\alpha 1\rangle - \beta 0\rangle$	XZ (X then Z)

Table 2.1: Bob's state after Alice's Bell measurement and the required correction.

Figure 2.2 shows a visual of the quantum circuit that performs the teleportation process

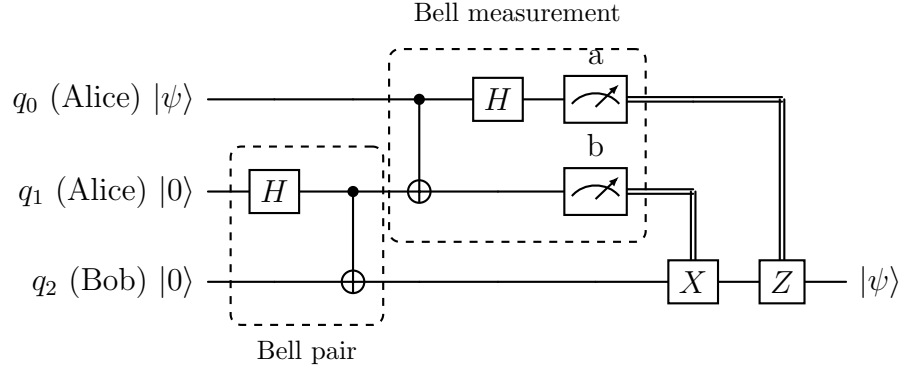


Figure 2.2: Quantum circuit for quantum teleportation

described above.

Note that this destroys both the original qubit Alice had, and also the bell pair. Also, this process does not violate the no-cloning theorem. Finally, because a classical communication channel is necessary, it is not possible to send data using this process faster than the speed of light.

2.2.4 Entanglement swapping

It is difficult to transfer qubit across large distance with a classical network like link. As distance increases, the efficiency of the channel decreases drastically. With no cloning theorem, the traditional repeater is unable to examine the qubit state. Therefore, it cannot directly amplify it.

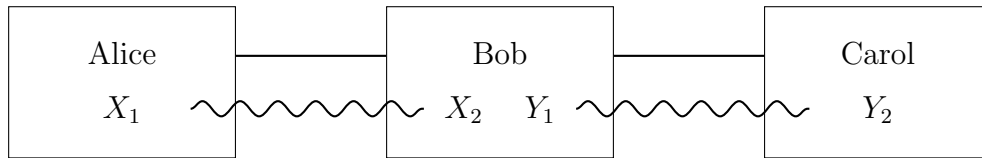


Figure 2.3: Before measuring X_2 and Y_1 at Bob's node

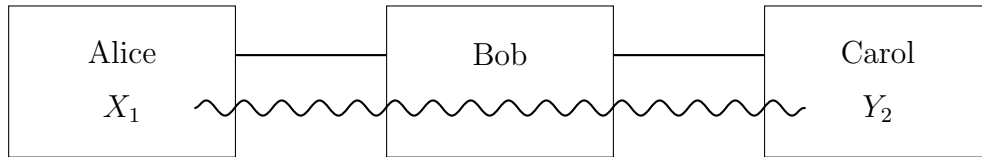


Figure 2.4: After measuring X_2 and Y_1 at Bob's node and sharing information

Figures 2.3 and 2.4 show a basic visualization of how entanglement swapping works between

three nodes (Alice, Bob, and Carol).

First, Alice and Bob share a $|\Phi+\rangle$ bell pair (X_1 and X_2), and Bob and Carol also share another $|\Phi+\rangle$ bell pair (Y_1 and Y_2). Alice and Carol do not interact with each other; the only connection is that Bob holds one qubit from each bell pair (X_2 and Y_1).

The combined qubit state is shown in the following equation (the state is written as $|X_1X_2Y_1Y_2\rangle$):

$$|\psi\rangle = |\Phi+\rangle_X \otimes |\Phi+\rangle_Y = \frac{1}{2}(|0000\rangle + |0011\rangle + |1100\rangle + |1111\rangle) \quad (2.19)$$

To perform the bell state measurement, a CNOT gate is applied with X_2 as the control qubit and Y_1 as the target qubit. This creates a relationship between the two bell pairs. After the CNOT operation, the following state is obtained:

$$\frac{1}{2}(|0000\rangle + |0011\rangle + |1100\rangle + |1101\rangle) \quad (2.20)$$

Finally, the Hadamard gate can be applied on qubit X_2 , making the following substitution:

$$|0\rangle_{X_2} \rightarrow \frac{|0\rangle + |1\rangle}{\sqrt{2}}, |1\rangle_{X_2} \rightarrow \frac{|0\rangle - |1\rangle}{\sqrt{2}} \quad (2.21)$$

This obtains the following state:

$$\begin{aligned} |\psi_2\rangle &= \frac{1}{2} (|0000\rangle + |0011\rangle + |1100\rangle + |1101\rangle) \\ &= \frac{1}{2\sqrt{2}} \begin{bmatrix} |0\rangle_{X_1} (|0\rangle + |1\rangle)_{X_2} |0\rangle_{Y_1} |0\rangle_{Y_2} \\ + |0\rangle_{X_1} (|0\rangle + |1\rangle)_{X_2} |1\rangle_{Y_1} |1\rangle_{Y_2} \\ + |1\rangle_{X_1} (|0\rangle - |1\rangle)_{X_2} |1\rangle_{Y_1} |0\rangle_{Y_2} \\ + |1\rangle_{X_1} (|0\rangle - |1\rangle)_{X_2} |0\rangle_{Y_1} |1\rangle_{Y_2} \end{bmatrix} \\ &= \frac{1}{2} [|00\rangle_{X_2Y_1} |\Phi^+\rangle_{X_1Y_2} + |10\rangle_{X_2Y_1} |\Phi^-\rangle_{X_1Y_2} + |01\rangle_{X_2Y_1} |\Psi^+\rangle_{X_1Y_2} + |11\rangle_{X_2Y_1} |\Psi^-\rangle_{X_1Y_2}] \end{aligned} \quad (2.22)$$

Now, Bob can measure the two qubits that it holds (X_2 and Y_1). After this, X_1 and Y_2 will become entangled. The results of the measurement will determine which Bell state that Alice and Carol's new bell pair is in. Carol can make the correction on her qubit based on

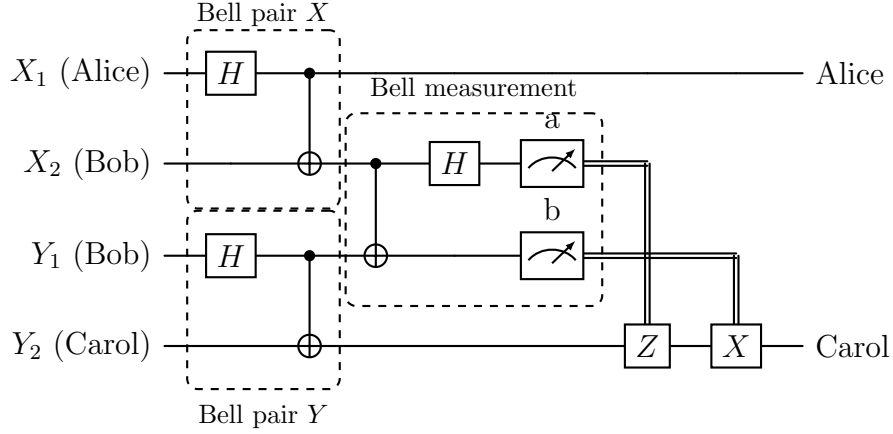


Figure 2.5: Entanglement swapping of 2 Bell pairs.

the result according to Table 2.2 to get the original $|\Phi+\rangle$ bell pair back.

X_2Y_1	Bob's state	Correction
00	$ \Phi+\rangle$	I (no corrections)
01	$ \Psi+\rangle$	X
10	$ \Phi-\rangle$	Z
11	$ \Psi-\rangle$	XZ (X then Z)

Table 2.2: Bell state that Alice and Carol hold after Bob performs BSM.

Figure 2.5 shows a circuit diagram for entanglement swapping between the 4 qubits.

The BSA node, described in Section 3.3.3, is used to perform entanglement swapping in a photonic quantum network.

2.2.5 Network Architectural Planes

A quantum network takes on a 4 plane architecture [39]:

1. Control Plane
2. Data Plane
3. Quantum Plane
4. Management Plane

The control plane is responsible for routing a data packet in a network. Each router has a routing algorithm installed that determines which path is the most optimal for a packet to travel to reach its destination. This is similar for both a quantum and a classical network. Even for a quantum network, the control signals will remain classical.

The data plane is responsible for delivering the actual data to the destination according to the routing information and the protocol which the control plane maintains and updates. For a classical network, this plane is used to send the actual data packets over the network. However, for a quantum network, the data plane acts as a classical data link to send supplementary information, such as RPC commands for interacting with hardware, and RuleSet events for qubit-level operation.

However, the quantum plane is what makes a quantum network distinctive. The quantum plane handles the transmission of qubits and entangled pairs within the network. It can be implemented in multiple ways. For example, the testbed in Moonshot Nagayama Project Laboratory (see Section 3.1) implements the quantum data plane using photons flowing in free space and fiber optic cable.

Finally, there is also a management plane for administrative operations, such as configuring the hardware in the network.

2.2.6 Architecture of quantum network

RFC 9340 ("Architectural Principles for a Quantum Internet") defines the basic components that make up one kind of quantum network [17].

Quantum repeaters will be the most important part of a quantum network and are also the core building blocks of one. RFC 9340 defined a repeater as having the following four functionalities:

- Creates an entanglement between two neighboring nodes
- Extends the range of entanglement beyond the neighboring nodes to distant nodes through entanglement swapping
- Performs distillation to ensure the fidelity of the entangled pairs
- Perform routing throughout the network and other management functions

A simple network with two nodes and one repeater is shown in Figure 2.6.

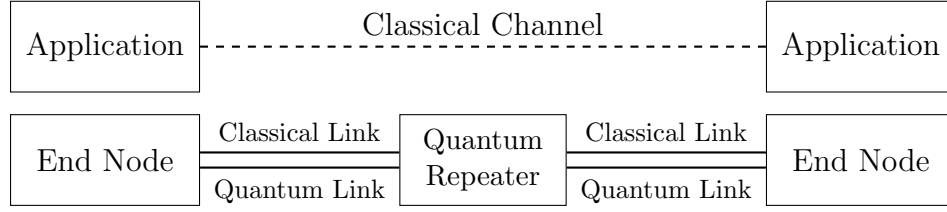


Figure 2.6: Simple quantum network with 2 nodes and a repeater.

When the two applications on each end node need to use an entangled qubit pair, first they communicate via the classical channel to configure and transfer any prerequisite information. Then, the classical channel between the repeater and each end node is used to coordinate resources and find routing procedures. Finally, it will generate entangled pairs and perform entanglement swapping using both the classical link and the quantum link available in the network. Once the entangled pair is ready, it will be made available to the application for use.

2.2.7 Quantum data center network (QDCN) and Q-Fly

A quantum network will not be very useful if there are only two end-nodes. Therefore, various structures for QDCN are proposed to organize network nodes structurally so that the network can be efficiently scaled in a data center network.

One such example of QDCN is called Q-Fly. Q-Fly is a scalable architecture that organizes nodes into groups [25]. Each group contains several quantum end-nodes and BSAs (see Section 3.3.3). These are connected by a large switch. The switch will give direct access between the BSAs and the quantum end nodes within that group. Inter-group communication can be done using the several outbound links that are also connected to the large switch.

There are 3 types of Q-Fly networks:

1. Single-path quasi-half duplex (SPHD) - Each group only shares 1 optical link.
2. Dual-path quasi-half duplex (DPHD) - There are 2 parallel optical links between groups.
3. Dual-path quasi-full duplex (DPFD) - Local connection no longer no longer needs to go through the switch, allowing a dedicated link to be reserved for both directions of

communication.

Figure 2.7 has a diagram for each of the 3 types of Q-Fly network.

SPHD networks reduces the smallest amount of links between networks, but will use more switch resources if two quantum end-nodes from far away groups need to share entanglement, and it is not possible to have bi-directional communication between 2 groups without using more switch resources. DPHD adds more links to the network, therefore, potentially reducing the amount of switch resources used by a connection. Finally, DPFD adds another set of direct connection between quantum end-nodes and the BSAs within the same group, eliminating the need to use a switch altogether for intra-group communication.

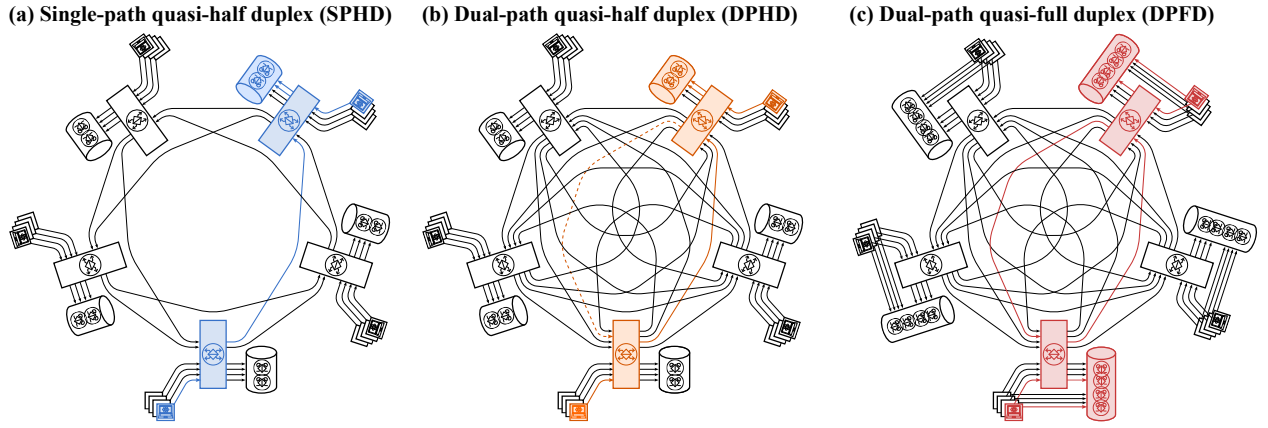


Figure 2.7: Types of Q-Fly networks. (Diagram from [25])

A portion of the Q-Fly network is being implemented on the Moonshot Nagayama Project testbed.

Chapter 3

Quantum Network Testbed Nodes

Moonshot Nagayama Project operates a laboratory in the KBIC building around the Shin-Kawasaki area. In the lab, experimentalists conduct experiments with different smaller components that form a quantum network. These experiments are separate and test only one specific component of the network.

The main goal of this project is to create an abstraction layer that brings our testbed beyond "simple individual experiments", but rather operates as a *network*.

On the testbed, the experimentalists had already arranged many devices to achieve different purposes in the network. Therefore, in order to build our logical node to run the network, we need our software to match the existing configuration on the testbed.

This chapter will cover the specifications of the devices that are already setup in the testbed and briefly cover the main goal of each node.

3.1 Testbed

The laboratory testbed is in the process of installing a small switching network, consisting of three memory-less, simple quantum end nodes, as well as a switching-BSA node. This can be seen as one of the small groups that form the Q-Fly network. This forms the foundation for implementing the Q-Fly network in a real world system. A diagram of the overall structure of the system being installed and the nodes used within is shown in [Figure 3.1](#).

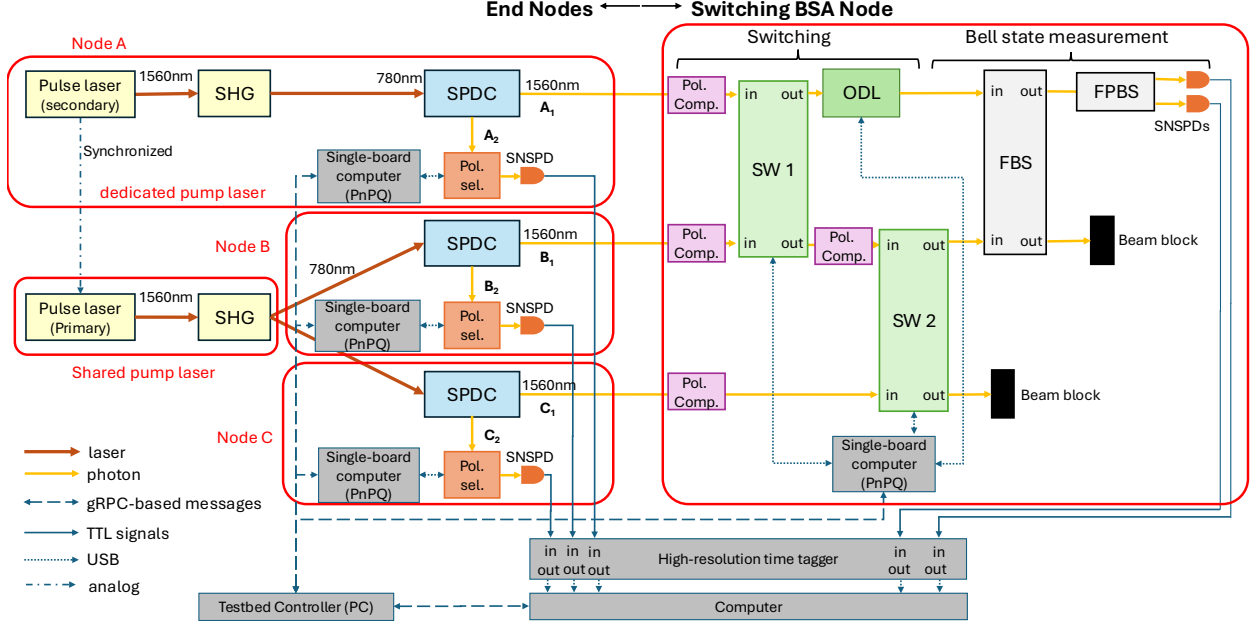


Figure 3.1: A single group within a Q-Fly network, consisting of three quantum end nodes and a switching BSA node. (Diagram from [25])

3.2 MEAS node

A MEAS node (or measurement node) receives a quantum state and performs a measurement in different bases. This enables the collection of statistics which is necessary to perform quantum state tomography.

A MEAS node that is used on the testbed receives photons traveling in free space. At the moment, there is only one type of MEAS node. However, in order to accommodate the addition of other designs of the MEAS node, this current design will be labeled Type A in `logical_node_service` API. The Type A MEAS node is built using the following hardware:

1. Quarter waveplate - Mounted onto a rotating waveplate mount (K10CR1, see section 4.2).
2. Half waveplate - Mounted onto a rotating waveplate mount (K10CR1).
3. Optical Collimator
4. Polarizing Beam Splitter
5. Detector - SNSPD and Multiharp (see section 4.6)



Figure 3.2: A single measurement node.

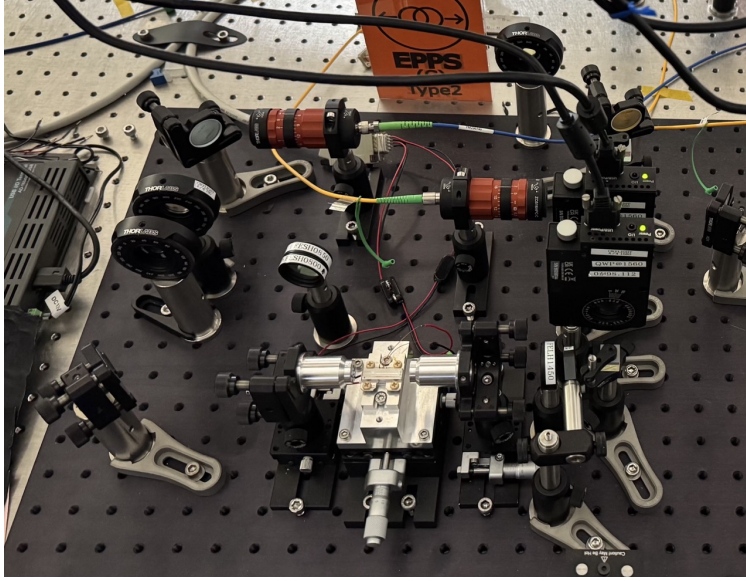


Figure 3.3: Image of a MEAS node connected to one output of the EPPS on the testbed.

These devices are arranged in the order shown in Figure 3.2. Figure 3.3 shows a picture of the MEAS node on the testbed, which is connected to one of the EPPS outputs. The 2 square shaped device are the waveplates that need to be operated in order to use it. The detector is relatively large; therefore, it is not included in the picture.

v	Basis	hwp	qwp
1	$ H\rangle$	45°	0
2	$ V\rangle$	0	0
3	$ D\rangle$	22.5°	45°
4	$ R\rangle$	22.5°	0

Table 3.1: Angles for waveplates for measuring various bases using a MEAS node. (Table adapted based on information from [12])

3.2.1 Quantum state tomography

Quantum state tomography is a technique for determining a quantum state experimentally. As mentioned previously, it is impossible to retrieve the full state of a qubit. Therefore, in

order to obtain information about the state, it is necessary to collect many samples of the qubit measured in different sets of bases and reconstruct the state statistically.

In order to perform state tomography on a qubit, measurements of 4 bases are required. Measurement data for each basis can be collected by setting both the half and quarter waveplate to the angles listed in Table 3.1 respectively, then totaling the number of detector clicks received over the period of time.

Using these data, it is possible to calculate the resulting density matrix. First, the total count of the photons can be obtained by adding the count for H basis and V basis:

$$N = n_H + n_V \quad (3.1)$$

Then, the three Stokes parameters can be calculated:

$$s_1 = \frac{n_H - n_V}{N}, s_2 = \frac{2n_D - N}{N}, s_3 = \frac{2n_R - N}{N} \quad (3.2)$$

Finally, using these Stokes parameters, the resulting density matrix can be calculated:

$$\hat{\rho} = \frac{1}{2} \begin{pmatrix} 1 + s_1 & s_2 - is_3 \\ s_2 + is_3 & 1 - s_1 \end{pmatrix} \quad (3.3)$$

3.2.2 Functionality of logical MEAS node

Within `logical_node_service`, the MEAS node will have a function that automates the process of measuring the 4 qubit bases. This includes repeating the process of rotating the waveplates and measuring using the detector for each basis. In the end, it will return a result with the 4 measurement results back to the client to perform the final calculation.

A pseudocode that implements state tomography for the MEAS node is in Algorithm 1.

Algorithm 1 Single-qubit state tomography on one MEAS node.

```
1: Setup clients
2: Initialize total count for all bases
3: repeat
4:   Rotate quarter-wave plate
5:   Rotate half-wave plate
6:   Start streaming from the detector
7:   while data are available from the detector stream do
8:     Add each received time tag to the total count
9:   end while
10: until all four bases have been measured
11: return Total count for all bases
```

3.3 Other logical nodes

Currently, at this stage, the MEAS node will be implemented within the `logical_node_service` package, while the other nodes described below will be for future projects for other students. The descriptions below reflect the current design and functionality of these nodes within the API definitions of the library, but are not finalized yet and may be subjected to changes.

3.3.1 EPPS node

An EPPS (entangled proton pair source) node, as its name suggests, produces entangled pairs of photon. This can be done using a process called SPDC (spontaneous parametric down conversion).

The input to an EPPS node is a continuously operating pulse laser that produces *pump photon*. Inside an EPPS there is a nonlinear medium such as a bulk crystal or a waveguide. For SPDC, inside the medium, some photon splits into pair of two photons of lower energy level, this pair of photons is called the idler photon and the signal photon. The idler and signal photons are entangled [40, 1].

There are two types of SPDC that are used on the testbed to build EPPS nodes: Type 0 and Type 2. Type 0 produces the state $|\Phi+\rangle$, and the signal and idler photons will be emitted at wavelengths 1540nm and 1580nm respectively. Type 2 produces the state $|\Psi+\rangle$, and both signal and idler photons are emitted at a wavelength of 1560nm [40].

The EPPS node does not have any programmable settings at the moment. However, it is

still necessary to include it in the control software so that it can have the entire picture of the nodes on the testbed. The output wavelength of the EPPS will also be tracked as a property of the logical EPPS node for reference purposes.

There is a temperature control setting for the EPPS node, but it cannot be controlled using a program at the moment; therefore, it will not be included as a function.

3.3.2 OSW node

The OSW (optical switch) node is one of the most important nodes in a network. It is the node responsible for performing the actual routing in the quantum network, delivering photons from one place to another based on their intended destinations.

The OSW switch consists of two parts: a controller and the hardware switches [40]. The controller is responsible for communicating with other OSW nodes in the network and other nodes (BSAs, end-nodes, etc.), in order to decide and reserve a path and configure the hardware to open the decided path [30].

An optical switch can be as simple as a 2x2 switch with two states, bar and cross. This can be achieved by simply using existing hardware such as the Thorlabs OSW22-1310E that is available on the Moonshot Nagayama Testbed (see section 4.5). However, a simple 2x2 switch may not be enough for many networks. For example, in the Q-Fly network example given in Figure 2.7, even the simple SPHD network with five groups uses 6x6 switches. Composite switches that combine many small switches, such as 2x2 switches, to form a large switch have been in the work all the way since the telephone network era [4]. It is also possible to do so for a quantum network. There are algorithms that can be used to operate multiple optical switches, causing them to appear as a single logical switch [16].

3.3.3 BSA node

The BSA node performs the Bell state analysis. An important use case for this is to perform entanglement swapping within a MIM (Memory-Interface-Memory) network, where 2 photons meet within the BSA in the middle as shown in Figure 3.4. This enables entanglement swapping in the network (see section 2.2.4) [9]. The goal of this type of network is to route 2 quantum end-nodes to the same BSA node, allowing entangled qubits to be distributed between the 2 end-nodes [30].

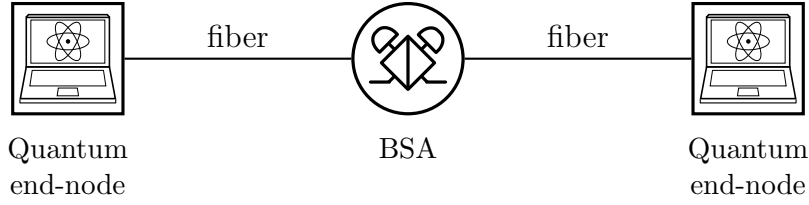


Figure 3.4: Two quantum end-nodes connected through a Bell State Analyzer (BSA).

The specific design for the BSA node on the testbed is still a work-in-progress task and will not be covered by the scope of this project. However, the foundation that this project builds will make it possible to build the BSA node. This will become an opportunity for a separate project in the future.

Chapter 4

Testbed Hardware

This chapter introduces various hardware that is used in the testbed, which is used to build logical nodes for our quantum network introduced in Chapter 3. These devices are supported by our software and will be used by the testbed logical node package in order to operate on the network. More details about the software will be introduced in chapter 5.

4.1 Thorlabs APT Protocol

Many Thorlabs devices that we have use a proprietary protocol called APT for communication [35]. In this protocol, each command is divided into a message which can be sent and received via a serial connection to the device itself.

Each message is divided into a header and a message body. Not all messages require a body. The message body can also have different lengths depending on the purpose of the message and how many parameters it requires.

The message header contains information such as the message code that defines the action, as well as information about the destination and source of the message. There are also a byte for the channel identifier that some devices will use.

For example, a message that moves a specific motor channel on a waveplate (see section 4.2) will be 12 bytes. In addition to the header, the channel identifier as well as the angle (in steps) to move the device are required.

Table 4.1: Supported Commands for Thorlabs APT Devices

Command	Description
Absolute Move	Move the device to a specified position.
Jog	Move the device in the specified jog direction by the configured jog step size.
Home	Calibrate the device and return it to the home position (typically 0 steps).
Is Homed	Return whether the device has been homed.
Identify	Flash the onboard LED to identify the device.
Get/Set parameters	Customize behavior of move, jog, and home.

Messages such as device identification do not require a message body as there are no parameters.

The Thorlabs devices do not use the metrics unit. They use their own proprietary unit called *steps*. Different devices have different definitions of the unit and the scale for a step. For example, 1 degree of rotation on the K10CR1/2 is 1370/170 steps, and 1mm on the ODL100 is 2000 steps.

4.2 Motorized Cage Rotator

The K10CR1 and K10CR2 motorized cage rotator is a device that comes with a precision motor that can rotate a cage frame. A polarization waveplate can be installed inside the cage and can be rotated to change the polarization of light passing through the lens [33]. Figure 4.1 shows the K10CR2, the circular hole within the rotator is where the polarization lens should be installed.

Table 4.1 shows the functions that are supported in the testbed software for this device.

4.3 Optical Delay Line

The optical delay line is a simple mechanical device that contains a set of mirrors mounted on a rail. The mirrors can move forward and backward along the rail, controlled by a precision motor. This can increase and decrease the length a photon has to travel, therefore physically creating a delay in the arrival time for photons going through that link [32].



Figure 4.1: Image of Thorlabs K10CR2. (Image from [33])

There are two types of device that are available on the testbed and supported by the software stack: Thorlabs (operated by the KBD101 motor controller) and OzOptics 650ml.

The Thorlabs optical delay line cannot be controlled directly using a computer. The commands has to be sent through a motor controller, such as the KBD101. The KBD101 supports the same sets of commands as the waveplate, and is shown in Table 4.1 [36]. Figure 4.2 shows an image of the ODL100 connected to the KBD101 motor controller.

4.4 Motorized Polarization Controller

The Thorlabs motorization polarization controller also acts as a waveplate, similar to the functions of the K10CR1/2 (section 4.2), but for photons inside fiber optics rather than in free space [34]. There are two versions: the MPC320 that has three channels and the MPC220 that has two channels. Figure 4.3 shows the MPC320.

Similar to the Thorlabs devices introduced previously, the motorized polarization controller also supports the functions in Table 4.1. Except that every motor arm has their own channel, enabling each arm to be controlled separately.



Figure 4.2: Image of Thorlabs K10CR2 with Thorlabs KBD101 motor controller. (Image from [32])



Figure 4.3: Image of Thorlabs MPC320 with 3 channels. (Image from [34])

4.5 Optical Switch

Optical switch is a controllable switch device that can redirect the flow of photon. Thorlabs provides the OSWxx-yyyyE series of switches (where **x** and **y** represent the specific model number within this category of devices). Switches can be of various shapes. The testbed uses OSW22-1310E, which is a 2x2 switch that has two states: cross and bar [31].

Optical switches are crucial in a quantum network. They are the node that enables routing between different quantum endpoints or between other optical switch nodes.

Smaller switches can also be configured to create more sophisticated switches with more inputs and outputs. Refer back to section 3.3.2 for more details.

4.6 SNSPD and TDC

SNSPD and the *Multiharp* are used in the lab to perform photon counting as part of the measurement node.

SNSPD (Superconducting Nanowire Single-Photon Detector) contains a thin superconducting wire that is cooled by a strong refrigerator down to cryogenic temperature below 2.5K. There is a small bias current that flows through the wire. When a photon hits the wire, the wire will receive some of the energy, causing the wire to be slightly resistive. This creates a short pulse that will be amplified and can be picked up by a computer. [26]

A TDC (time-to-digital converter) receives the detected pulse from the SNSPD, assigns a precise timestamp, and sends it to a receiving computer or storing it in a database for later use. The testbed uses a model of TDC called Multiharp by PicoQuant [24].

This setup will become the foundation of the measurement node and the BSA in the network, which requires measuring photon as one of their main functions.

Chapter 5

Testbed Software Architecture

Before building `testbed_logical_node`, a backbone support architecture must be created. For the past two years and more, much work has been put into this foundational architecture, from creating the support to control physical devices (both Thorlabs and Multiharp) using Python and Rust to RPC interface that allows devices to be controlled remotely. This chapter will explain a bit about the design decisions taken while building these and how they contribute to building the logical node software.

The software stack that will run on the testbed is shown in Figure 5.1.

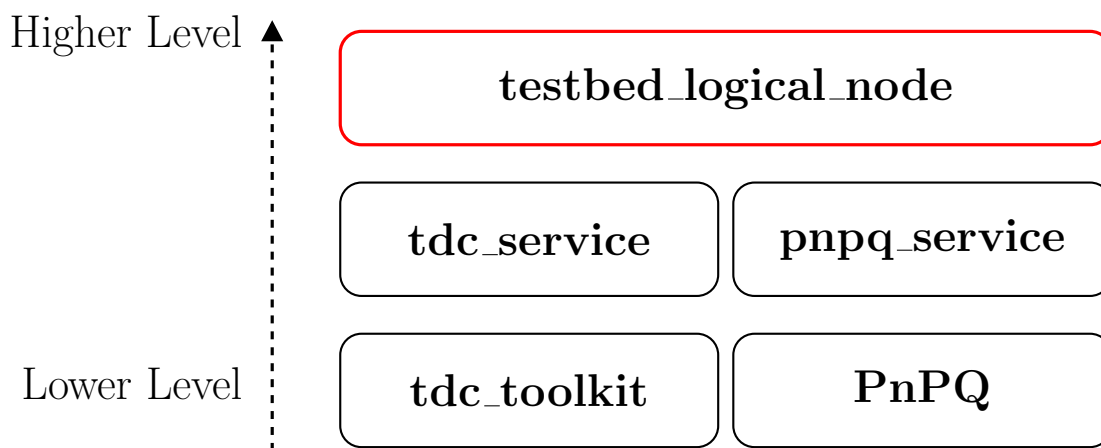


Figure 5.1: General architecture of the testbed software.

5.1 General Architecture

As detailed in Figure 5.1, before working on the proposed library package - `testbed_logical_node`, many dependencies need to be created first to support lower level operations. These dependencies have been created with the help and support of many colleagues in the laboratory. There are many features that are left to implement, creating opportunities for younger students to work on as their projects.

These software can be split into three layers, each layer adds a level of abstraction. The bottom layer interfaces directly with the hardware devices. They have to run on the computer that has the hardware directly connected. The middle layer runs on the same hardware as the bottom layer, and provides a remote interface for the bottom layer software, enabling devices to be controlled remotely.

Finally, the top layer is where `testbed_logical_node` will run. It'll interact with the middle layer software running on devices on the testbed. More details about `testbed_logical_node` will be introduced in chapter 6.

5.2 Driver Packages

Devices used on the testbed such as Thorlabs and the Multiharp already come with software that can operate the device. However, these softwares are generally aimed at experimentalists who may want to control these devices manually. These applications generally only offer a GUI interface with minimal customizability. Therefore, there is a need to develop Python software packages

5.2.1 pnpq

PnPQ is short for "Plug-and-Play Quantum". It is the software that interacts with the optical hardware on the testbed. It is part of the bottom layer of the software architecture. The goal of this software is to make it easy to control hardware on the testbed using Python scripts or Rust programs. Although the software does not enable true "plug-and-play" behavior, it is very easy to setup the device in a program via the serial number. It runs on the Raspberry Pi to which the hardware devices are directly connected.

PnPQ currently supports four types of device:

1. Optical delay line (OzOptics and Thorlabs)
2. Motorized polarization controllers
3. Motorized rotation mount (for waveplates)
4. Optical switches

PnPQ is available in both Python and Rust. At the time of writing, the Python version is already published on PyPI (Python Package Index) [28]. However, the Rust version is still under development.

As introduced in chapter 4, devices 1-3 communicate via Thorlabs's APT protocol. Therefore, PnPQ implements a helper behind the scene to translate between Python or Rust objects into an APT message that can be sent via serial connection to the device.

PnPQ provides a blocking API. This means that functions will only return until the intended function has been completed on the hardware device. This decision has been made because the performance gain of parallel movements is not a problem for the experiments being used within the lab, while doing it this way improves reliability of the system.

PnPQ also leverages the package `pint` for Python version and the crate `uom` for Rust version. By using this library, it enables the user to pass any compatible unit of their liking rather than a fixed unit. In addition, it prevents the user from passing incompatible units, such as a length unit such as meters to a place where angle unit such as degrees is expected, minimizing errors on the user side.

PnPQ wraps the driver for each device into its own type, as well as a helper connection type for handling the connections for APT based devices. For example, the code snippet below shows an example of initializing the MPC320:

```
1 with AptConnection(serial_number="xxxxxxx") as connection:
2     device =
        PolarizationControllerThorlabsMPC320(connection=connection)
3     device.move_absolute(ChanIdent.CHANNEL_1, 90 *
        pnpq_ureg.degree)
```

PnPQ also supports stub devices that simulate API behaviors of real devices. Therefore, even if a real device is not available, it can still be used for testing and development. Stub devices provide an identical function interface compared to the real device.

PnPQ is available for download using `pip install` and is also available on [Github](#).

5.2.2 tdc_toolkit

`tdc_toolkit` is a Rust library that can talk to time-to-digital converters. Currently, it only supports the Multiharp MH160 that is currently used in the laboratory testbed. It runs on a mini-PC that has the Multiharp directly connected.

`tdc_toolkit` configures the Multiharp device using a `json` configuration file and allows the measurement duration to be specified. The command line tool is the default way for interacting with this software:

```
1 tdc_toolkit record --device-config=multiharp160_no_filters.json
   --duration 2s
```

This will start a measurement session on the Multiharp device with the specified configuration, process the data through a pipeline, and store the output into a parquet file.

To use this remotely, simply connect to the device

In addition to the command line tool, it is also possible to use the library directly in Rust. The library will provide access to the individual pipeline that can be extended and integrated into other systems. `tdc_service` will leverage this to create the streaming functionality.

`tdc_toolkit` is available for download on the Rust Package Registry as a crate and is also available publicly on Github [37].

5.3 Service-oriented architecture

The entire software architecture, including both `logical_node_service` and its dependencies, follows an architecture called *service-oriented architecture*. Service oriented architecture separates different functionalities into their own microservice. There are many benefits to this architecture, especially when it comes to creating a network control service [6].

5.3.1 Benefits of microservices

Firstly, each microservice is deployed separately and is independent of each other. For example, a separate running copy of `pnpq_service` is installed on each Raspberry Pi. Therefore, if one Raspberry Pi goes offline, it will only affect the physical hardware connected to that single Pi (and the logical node that depends on it). This allows for isolation of failed component, without taking the entire quantum network offline.

Next, the microservice ensures modularity. This can simplify development and keep the code easy to extend and add additional functionalities. For example, if a separate lab has a different design for a node with a different hardware, they can simply add the hardware driver to `pnpq`, then create a new type of node that interacts with that particular hardware to achieve the same functions.

Finally, microservice based architecture creates an open interface, enabling users of this platform to use any tools they like. gRPC is platform-independent, meaning it supports a variety of programming languages, as long as there is a library that can act as a client.

5.3.2 Schema First Development

During the development of this software package, a technique called *schema first development* was used.

Schema first development refers to the application development process in the API schema is created first before the actual implementation begins. In the case of `testbed_logical_node`, each node will first have the compositions and functions defined before the implementation begins. The schema design will also be confirmed with the experimentalists on the testbed to ensure that it matches the hardware setup that is currently present on the testbed.

This technique was used because it allows the overall structure of the service to be known in a stage where it is easy to review and edit. Once implementation starts, it may be difficult to change the function interface or the underlying object model due to the underlying complexity.

5.4 gRPC Services Packages

gRPC provides a remote function call interface. The purpose of this framework is to allow a client to call functions on a server as if it is a local function. This simplifies the development of distributed systems.

gRPCs allow 4 types of RPC [8]:

1. Unary RPC - Client sends single request, server returns single response.
2. Server streaming RPC - Client sends single request, server streams sequence data.
3. Client streaming RPC - Client streams data for the request, server returns a single response at the end.
4. Bidirectional streaming RPC - Both client and server streams request and response.

These 4 types of RPC support flexibility in designing the APIs for both the gRPC wrapper for `pnpq` and `tdc_toolkit`, as well as creating the actual logical node. Most of the blocking device functions in `pnpq` use a unary RPCs, and the streaming RPCs simplify the process of sending back large amounts of time tag data to the client, as Multiharp also returns results in stream mode.

gRPC supports many programming languages and frameworks, making it a very flexible option to use in an experimental scenario. The client and the server do not have to use the same library or even be written in the same language. For example, Figure 5.2 shows an example of where both `logical_node_service`, which is written in Rust, and a small test Python client developed by experimentalists, can make the same call to `pnpq_service` using the gRPC client library available for each programming language.

A system that uses gRPC requires the schema to be defined first using languages such as Protobuf (Protocol Buffers). Google created Protobuf for designing structured APIs.

After the schema has been defined, the implementation of the service for both the client and the server can be automatically generated. In Rust, this functionality is provided by the `tonic` crate. Server-side generation produces template code that contains service traits that must be implemented by the application. Once these trait implementations are provided, the service becomes available for handling remote procedure calls.

On the client side, the generated code includes functions that provide an interface for invoking remote procedures. Function calls issued through these stubs are serialized and transmitted to the server for execution, and the corresponding responses are returned to the caller, providing an interface that closely resembles invoking a local function.

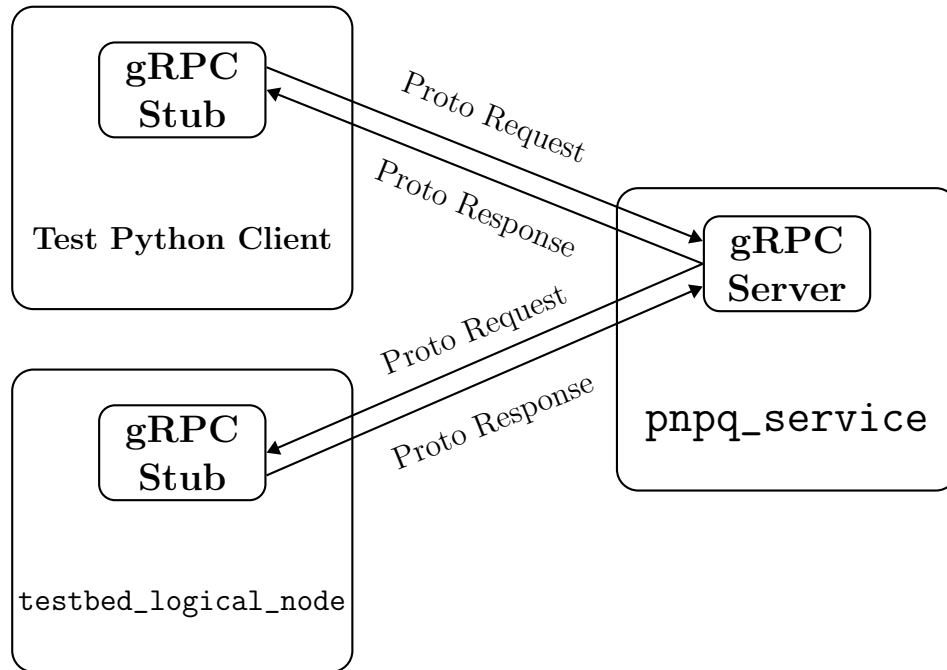


Figure 5.2: Communications between `pnpq_service` and different types of clients

5.4.1 `pnpq_service`

`pnpq_service` implements the middle layer functionality of our software architecture for PnPQ by providing a gRPC interface that enables remote operation of devices using PnPQ.

Each device gets its own API that roughly matches the corresponding API for that device within PnPQ. For example, the function `move_absolute` for the MPC referenced in section 5.2.1 has a corresponding API definition like the following:

```

1 service PolarizationControllerThorlabsMpcService {
2   rpc MoveAbsolute(MoveAbsoluteRequest) returns
    (MoveAbsoluteResponse);
3 }

```

5.4.2 tdc_service

`tdc_service` contains three main functions.

1. Get device information
2. Record to file
3. Record and stream data

The first two functions are a simple wrapper of existing functions in the command line tool provided by `tdc_toolkit`. The last is a unique function to this library. gRPC supports streaming of data over a period of time. Therefore, we can leverage this capability to continuously stream real-time measurement data over the network to the client, which will be useful when we need to enable responsive behaviors within each logical node.

Chapter 6

Testbed Logical Node

The `testbed_logical_node` package sits on the top layer, above all the software introduced previously in chapter 5. It provides a high level of abstraction, eliminating the need for experimentalists to operate on individual devices altogether. Instead, they will operate at the network node level. A node here refers to the a logical group of hardware devices that works together to perform a high level function.

6.1 Design Overview

In order to make `testbed_logical_node` as useful as possible, not only within the testbed of Moonshot Nagayama Project, but also to other laboratories, the software needs to take on a modular structure, and can be easily extended to add nodes with different designs with different physical hardware.

Similarly to the software in the middle layer that requires remote control, `testbed_logical_node` also uses gRPC as its interface, both for configuration and control. This can maximize compatibility, as users can use any compatible client they wish.

In order to achieve this, the software foundation will contain the following three components.

6.1.1 Physical device manager

This component serves as the foundation for building logical nodes.

Currently on the testbed, there are many devices that are in different locations on the testbed. This makes it difficult to connect all devices to the same computer controller. This means that in order to build logical nodes, it is necessary to communicate to many different Raspberry Pis as well as to the PC with the Multiharp connected to it. Since each of them runs a separate copy of `pnpq_service` or `tdc_service`, `testbed_logical_node` needs to gain the ability to talk to different gRPC servers concurrently. Therefore, the need for a device manager increases.

This component can be configured with a list of hosts that run device services such as `pnpq_service` or `tdc_service`. It will attempt to discover the specified hosts automatically, load the hardware information, and set up internally to allow communication with the connected devices.

This physical device manager will also abstract the physical location of the hardware. Once the device is discovered, an alias can be set to it, and this component will automatically send requests to that device alias to the corresponding hardware on the corresponding control computer.

6.1.2 Logical node manager

On top of the physical device manager is the logical node manager. This component is responsible for initializing each logical node, keeping track of which device is used by which node, and storing them during runtime.

A logical node in the context of this software package is an instance that represents a set of hardware on the testbed that is arranged according to the specifications described in chapter 3. It can contain 0 or more physical devices, and also any necessary parameters to keep track of the device.

Each logical node will also get a name to identify it within the network, similar to how devices in classical networks can be identified using an IP address.

6.1.3 Experiment manager

Since this software is still initially going to be used by many experimentalists, we will add a component to support the tracking of experiments in the package.

This component will be very basic, with one function that allows the user to set an experiment name for that experiment session. This name will appear in the log files of any subsequent log messages that are generated as part of this experiment. There are no requirements on the formats for what the name is and there will be no checks in place in the API before updating the name in the system. At the moment, it is up to the users to ensure that the name is set and to ensure its integrity throughout the experiment process.

6.1.4 Documentation

Since Protobuf is the language used during schema development, packages are available to generate documentation while development is in progress.

Documentation for a specific node can be written in the comments above the declaration of each Protobuf RPC function. Then, using a plugin to `protoc`, documentation of various formats (such as HTML) can be generated automatically. This file can then be used as a supplement to provide functional information in addition to the written specifications of the logical nodes.

6.2 Logical Node Designs

The schema of each logical node used in the package `testbed_logical_node` at the time of writing this thesis can be found in [Figure 6.1](#).

At the layer for this system, each node will operate independently, with limited capabilities for communicating with the other. However, automation algorithms such as the RuleSet system and routing protocols can be built on top of this software to enable nodes to communicate with each other.

Detailed information on the design and function of each node can be found in [Chapter 3](#).

Note that the available functions for the BSA and Switching BSA node have not yet been decided and are available as a project in the future.

These five types of nodes are used in the current testbed design. However, the system is not limited to supporting only these types of nodes. If a different laboratory wants to use this platform and they have their own custom node, it is relatively simple to add a new service

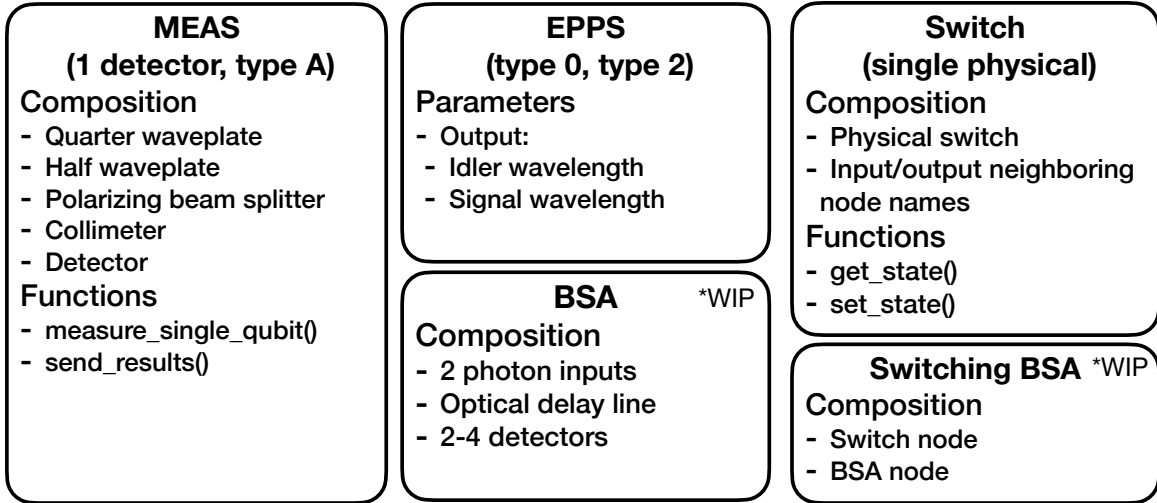


Figure 6.1: Schema design of logical nodes.

that implements the functionality of the new node.

6.3 Implementation Details

The software will be implemented using Rust as an executable Cargo package.

First, the APIs for this package is defined using the Protobuf language mentioned above, similar to its dependencies. Then, a Cargo crate called `tonic` is used as a foundation to provide the gRPC service for the library.

Since this is a service oriented software, there will not be use of configuration files to setup the service. Instead, there is an admin service that handles the creation and setup of logical nodes, as well as any administrative functions that are usually handled using configuration files.

A `sqlite` database will be used to store these configurations. At the moment, in memory database is used as persistency is not a necessity at the moment. Users are required to configure this service to configure all nodes before using it.

Chapter 7

Evaluation

This chapter will evaluate this project using several factors, analyzing the strength and weakness in each aspect, as well as identifying areas for improvements for both the development process and the final software package.

7.1 Functionality Check

The first version of `logical_node_service` is designed to work on the Moonshot Nagayama Project testbed. Therefore, as part of the requirement, it needs to be able to perform a single qubit state tomography using one of the hardware MEAS node setup on the testbed.

The MEAS node used in the testbed setup has 2 pairs of HWP and QWP connected to a single Raspberry Pi via an USB hub, in which one of the pair will be used in this experiment. This MEAS node is also connected to the PicoQuant Multiharp that's inside the lab.

First, all dependencies services (`pnpq_service` and `tdc_service`) are installed on the respective computers. Then, `logical_node_service` is run locally on the laptop used for development. The laptop is connected to the same network as both the Raspberry Pi and the computer controlling the Multiharp, enabling them to connect via gRPC.

Table 7.1 shows the data obtained from the service after running the `measure_single_qubit` function.

With this data, it is now possible to calculate the final density matrix. First, calculate a

v	Basis	Result
1	$ H\rangle$	492303
2	$ V\rangle$	480891
3	$ D\rangle$	476982
4	$ R\rangle$	471624

Table 7.1: Detector count returned from the MEAS node for each bases.

total count for normalization by adding the count for H and V bases:

$$N = 492303 + 480891 = 973194 \quad (7.1)$$

Then, the three Stokes parameters can be obtained:

$$\begin{aligned} s_1 &= \frac{492303 - 480891}{973194} = 0.011726, \\ s_2 &= \frac{2(476982) - 973194}{973194} = -0.019760, \\ s_3 &= \frac{2(471624) - 973194}{973194} = -0.030771. \end{aligned} \quad (7.2)$$

Now, the final density matrix can be obtained:

$$\begin{aligned} \hat{\rho} &= \frac{1}{2} \begin{pmatrix} 1 + 0.011726 & -0.019760 - i(-0.030771) \\ -0.019760 + i(-0.030771) & 1 - 0.011726 \end{pmatrix} \\ &= \begin{pmatrix} 0.505863 & -0.009880 + i 0.015386 \\ -0.009880 - i 0.015386 & 0.494137 \end{pmatrix} \end{aligned} \quad (7.3)$$

With this, the state tomography process has been completed.

7.2 Quality Analysis

In addition to checking the required functionalities, the logical node service itself will also be evaluated using the criteria in the following subsections, with its strengths and weaknesses discussed for each, where applicable. ISO/IEC 25010 contains one such definition of criteria [11]. These criteria include aspects such as usability, extensibility, and more.

This section will analyze `logical_node_service` using some of these criteria that are relevant to the requirements of the software, highlighting the advantages and trade-offs of adopting a logical node level testbed software architecture. In addition, the resulting software will be compared with a traditional device-centric operation approach to assess how effectively the logical node service addresses its limitations. The criteria used for the analysis of this system are shown in Table 7.2.

Criterion	Description
Usability	How simple is this software to use compared to existing systems?
Reusability	How well does it handle different hardware configurations?
Extensibility	How easy is it to add new logical nodes to the system?
Portability	How well does it work in different testbed environments?

Table 7.2: Evaluation Criteria

7.2.1 Usability and simplicity

Simplicity is the biggest advantage that `logical_node_service` brings over the former device-centric approach. By introducing abstraction, many of the complexities required to perform operations and experiments on the testbed, such as state tomography, were removed.

Previously, to run a process like state tomography using the device-centric approach, it was necessary to initialize each device separately. If the device was connected to multiple Raspberry Pis, it was also necessary to initialize multiple clients to handle each device. Each type of device also has its own service. Combined, this creates many complexities that need to be managed, which is unsuitable for automation in a testbed. Figure 7.1 shows the tree diagram of the two services that needed to be managed to even run a single in a device centric manner.

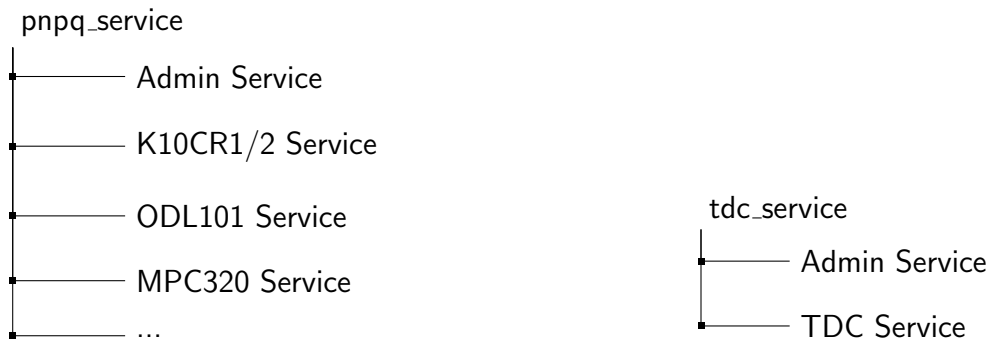


Figure 7.1: Service architecture tree for `pnpq_service` and `tdc_service`.

After the devices are initialized, to perform complex processes on the testbed, multiple hardware needs to be operated together. Device centric operation requires an API call for every move. For example, to perform state tomography as shown in Algorithm 1, there are three API calls that are necessary for every bases:

1. Rotate quarter waveplate (via `pnpq_service`)
2. Rotate half waveplate (via `pnpq_service`)
3. Measure using Multiharp (via `tdc_service`)

Therefore, to measure four bases, twelve API calls are required.

By abstracting the behavior of each node, only one API call is necessary. This made the call site much simpler, which is important when higher level routing and automation protocols will be operating on these testbed nodes.

By introducing simplicity in the use of the software, it is no longer necessary to know the technical APIs required to program hardware devices. This makes the testbed easier to use and more accessible to hardware experimentalists.

However, simplicity usually comes with the loss of customization. In this case, advanced users who may want custom behavior for a node must now either implement the extension into `logical_node_service`, or resort to a library on a lower level.

7.2.2 Reusability

Optimizing for reusability is another motivation for development `logical_node_service`. Currently, for a device-centric setup, if the hardware configuration changes, then the code will no longer be compatible.

For example, Figure 7.2 shows two different possible configurations in which the quarter and half waveplates can be set up. A device-centric code for Testbed A makes the assumption that both Raspberry Pis are connected to the same device. However, this means that the code cannot be reused for Testbed B, which has the two waveplates connected to different Raspberry Pis.

`logical_node_service` improves this by using its physical device management layer that removes the dependency on the physical location of the devices. This enables the code to

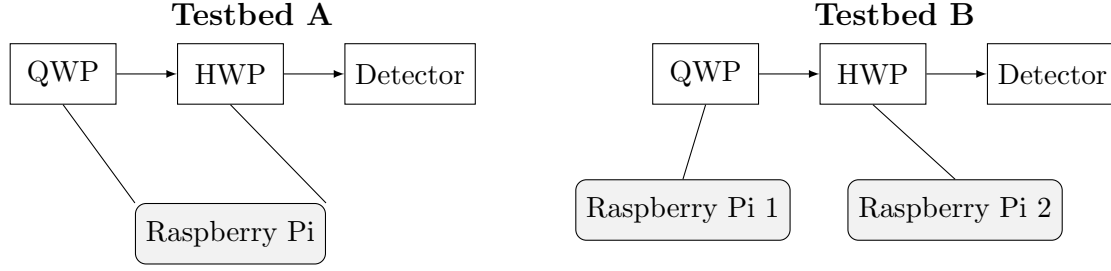


Figure 7.2: Comparison of two possible hardware configurations of one MEAS nodes on two different testbeds.

be reused between testbeds. The only part which needs to be changed is the hardware configuration, which is a one-time setup using an API within the Admin Service within the software.

7.2.3 Extensibility

Extensibility refers to the ability for the software to be extended to support new types of node designs and hardware. This is very important when designing an abstraction layer. Without this, a program will lose flexibility and, therefore, will have limited use cases.

For `logical_node_service`, the three part system allows new types of quantum network node as well as new physical designs of existing network node to be integrated with ease.

For example, suppose that a new type of MEAS node being added with a different set of hardware, such as a different model of waveplate. For a device-centric approach, almost the entire code base needs to be rewritten to support the new variant, and in some cases, the two versions of code for the two variants of MEAS node may be incompatible, requiring code-level change for high level applications as well.

However, since `logical_node_service` contains an abstraction layer for all hardware underneath a device and provides a node-level API. Therefore, when adding a new device, simply add a new implementation of the same API that uses the new device driver. This allows higher level programs to stay exactly the same, and immediately work with the new design.

7.2.4 Portability

Portability here refers to the ability for the same software to run on testbeds other than the one of Moonshot Nagayama Project.

Another major contribution of `logical_node_service`, in addition to the code and the software package, is the standardization of the practical implementation of nodes on a testbed. The result of this is a documentation that contains the exact set of devices required to create a node. If another testbed contains the same design, then it is possible to run `logical_node_service` directly to operate the nodes on the other testbed.

However, in order for this software to be portable, the nodes on the new testbed must strictly follow the definition of the node. If the design differs even slightly, a new type of node must be created to support the design. However, as described in Section 7.2.3, it is relatively simple to implement a new design into the software.

7.3 Overhead

Overhead is one of the main drawbacks of introducing abstractions into the testbed. The main contribution of this software comes in the form of abstraction. However, abstraction usually comes at a cost in terms of performance. In order to make a system easier to use and develop by programmers and experimentalists, an additional workload is generally introduced to handle the underlying work, providing the simplification, creating overhead [19].

`logical_node_service` replaces the device-centric operation of the testbed by providing a logical node abstraction layer in a centralized controller. However, this will increase the amount of traffic flowing through the classical control and data plane. If there are many operations that involve the detector, then many streams of data will be flowing through the network at the same time.

For example, if two MEAS nodes want to perform state tomography, then they will have to request the controller with `logical_node_service` to take measurement data from the detectors of the two nodes. Suppose that the detectors are on two separate Multiharp, then two streaming data will be going through the network to the controller, and the controller will have to forward both streams to the client making the tomography request. This requires the underlying classical network to have a high bandwidth.

7.4 gRPC service based system and event broker system

There are other systems which attempted to implement logical abstractions similar to the proposed system. One such example is the QUANT-NET Control Plane (QNCP) by the team at Lawrence Berkeley Laboratory [43]. QNCP starts with a hardware abstraction layer (HAL), similar to the proposed system. However, on top of the HAL, there is an event broker based system that handles communications within the system. For example, events such as RPC and hardware status updates are passed through the event broker. There are agents that handle applications in the system that can subscribe to any relevant events for that application.

There are many benefits that systems such as QNCP provide via an event broker. First, there are very loose coupling between the physical setup and the software. The controller does not need to care about who is consuming an event that it publishes, and the event broker is very effective at handling large number of simultaneous events that are emitted in parallel. In comparison, the proposed gRPC-based service, with its current function, always returns the result to a client. However, in the future, a multi-streaming APIs can be introduced for multiple clients to subscribe in order to perform complex tasks such as multi-qubit state tomography.

However, QNCP's benefit comes when the system is at scale. For a smaller network such as a quantum local area network (QLAN), an event broker system results in a large overhead due to the additional event bus component. The proposed gRPC service is much more lightweight, as the service only handles requests as they come, returning the results immediately. The lightweight nature of the proposed abstraction system enables simpler deployment to small scale network systems, as well as making debugging systems simpler.

Chapter 8

Conclusion

8.1 Contributions

This thesis presented a logical abstraction framework for controlling heterogeneous quantum networking nodes on a testbed. By abstracting physical hardware through a logical node layer and exposing the composition and functionalities through a service-oriented architecture, the proposed system decouples experiment code from the device specific approach.

A simple 1-detector logical MEAS node has been finalized and implemented in the system as a proof of concept. Then, an attribute-based quality assessment was used on the proposed solution. The result shows that, compared to the current device-centric approach, a logical abstraction layer provides improvements in many aspects such as simplicity, reusability, extensibility, and many more.

Although the abstraction added an extra layer of software to the existing stack that may require evolution to support new designs and new hardware capabilities, it provided a practical foundation for managing heterogeneous quantum networking nodes with its qubit-level API set. This is important for both experiments on the testbed and when introducing additional protocols such as routing control protocol within the quantum local area network.

8.2 Future works

8.2.1 Implement other nodes

Currently, only the MEAS node has been implemented within this project to demonstrate the feasibility of the concept. Furthermore, that MEAS node only has the capability to perform single qubit tomography. However, in order to make this project more useful, the MEAS node needs to be more complete. For example, a common use case on the testbed would be to support 2-qubit tomography. This requires the MEAS node to gain a new streaming API that streams the time tag so that the client can perform coincidence counting.

In addition, the system needs to support more types of logical nodes. In order to build a routing protocol on top of these logical nodes, all nodes (OSW, BSA, and EPPS) on the testbed must be supported.

8.2.2 Open source project and spread adoption

The final goal of this system is to simplify the development of quantum testbed experiment systems and provide standardized APIs to operate heterogeneous quantum nodes. Therefore, once all of the work-in-progress tasks are completed, the final goal would be to make this system open source and available for everyone to use.

8.2.3 Support higher level protocol

Once a good set of nodes is implemented, it is possible to implement a higher level protocol that automates resource allocation and routing within the quantum network. Examples of such protocols include an automatic configuration protocol for nodes and routing protocol that controls OSW (switch) nodes to open paths between quantum end-nodes and BSA node [29, 30]. Another direction for extension would be the implementation of a RuleSet system that automates the control of each node, eliminating the need to manually call APIs. In order to do this, the specific functionalities required by the protocol for each node need to be identified, then the implementation needs to be added for all.

Bibliography

- [1] Aliro. Quantum networking hardware: Entangled photon sources. White paper, Aliro, 2024. Accessed: 2026-07-05. https://www.aliroquantum.com/hubfs/White%20Papers/ALIRO%20White%20Paper_%20Quantum%20Networking%20Hardware_%20Entangled%20Photon%20Sources-2.pdf.
- [2] Abderrahim Amlou, Amar Abane, Mheni Merzouki, Lydia Ait Oucheggou, Zineb Maasaoui, and Abdella Battou. Automated network programmability using open-config yang models and netconf protocol. In *2023 20th ACS/IEEE International Conference on Computer Systems and Applications (AICCSA)*, pages 1–5, 2023. [doi:10.1109/AICCSA59173.2023.10479244](https://doi.org/10.1109/AICCSA59173.2023.10479244).
- [3] Charles H. Bennett, Gilles Brassard, Claude Crépeau, Richard Jozsa, Asher Peres, and William K. Wootters. Teleporting an unknown quantum state via dual classical and einstein-podolsky-rosen channels. *Phys. Rev. Lett.*, 70:1895–1899, Mar 1993. [doi:10.1103/PhysRevLett.70.1895](https://doi.org/10.1103/PhysRevLett.70.1895).
- [4] Charles Clos. A study of non-blocking switching networks. *The Bell System Technical Journal*, 32(2):406–424, 1953. [doi:10.1002/j.1538-7305.1953.tb01433.x](https://doi.org/10.1002/j.1538-7305.1953.tb01433.x).
- [5] Artur K. Ekert. Quantum cryptography based on bell’s theorem. *Phys. Rev. Lett.*, 67:661–663, Aug 1991. [doi:10.1103/PhysRevLett.67.661](https://doi.org/10.1103/PhysRevLett.67.661).
- [6] Martin Fowler. Microservices guide. <https://martinfowler.com/microservices/>.
- [7] Nicolas Gisin, Grégoire Ribordy, Wolfgang Tittel, and Hugo Zbinden. Quantum cryptography. *Reviews of Modern Physics*, 74(1):145–195, March 2002. [doi:10.1103/revmodphys.74.145](https://doi.org/10.1103/revmodphys.74.145).
- [8] gRPC.io Authors. Core concepts, architecture and lifecycle. <https://grpc.io/docs/what-is-grpc/core-concepts/>, May 2026. Last updated: May 11, 2026. Accessed: July 12, 2026.

- [9] Michal Hajdušek and Rodney Van Meter. Quantum communications, 2023. [arXiv:2311.02367 \[quant-ph\]](#).
- [10] S. L. N. Hermans, M. Pompili, H. K. C. Beukers, S. Baier, J. Borregaard, and R. Hanson. Qubit teleportation between non-neighbouring nodes in a quantum network. *Nature*, 605(7911):663–668, 2022. [doi:10.1038/s41586-022-04697-y](#).
- [11] International Organization for Standardization and International Electrotechnical Commission. ISO/IEC 25010:2023 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model, 2023. International Standard.
- [12] Daniel F. V. James, Paul G. Kwiat, William J. Munro, and Andrew G. White. Measurement of qubits. *Phys. Rev. A*, 64:052312, Oct 2001. [doi:10.1103/PhysRevA.64.052312](#).
- [13] Cody Jones, Danny Kim, Matthew T Rakher, Paul G Kwiat, and Thaddeus D Ladd. Design and analysis of communication protocols for quantum repeater networks. *New Journal of Physics*, 18(8):083015, 2016. [doi:10.1088/1367-2630/18/8/083015](#).
- [14] H. Jeff Kimble. The quantum internet. *Nature*, 453(7198):1023–1030, 2008. [doi:10.1038/nature07127](#).
- [15] Johannes Knörzer, Xiaoyu Liu, Benjamin F Schiffer, and Jordi Tura. Distributed quantum information processing: A review of recent progress. *arXiv preprint arXiv:2510.15630*, 2025. [doi:10.48550/arXiv.2510.15630](#).
- [16] Marii Koyama, Claire Yun, Amin Taherkhani, Naphan Benchasattabuse, Bernard Ousmane Sane, Michal Hajdušek, Shota Nagayama, and Rodney Van Meter. Optimal switching networks for paired-egress bell state analyzer pools. In *2024 IEEE International Conference on Quantum Computing and Engineering (QCE)*, page 1897–1907. IEEE, Sept 2024. [doi:10.1109/qce60285.2024.00219](#).
- [17] Wojciech Kozłowski, Stephanie Wehner, Rodney Van Meter, Bruno Rijsman, Angela Sara Cacciapuoti, Marcello Caleffi, and Shota Nagayama. Architectural Principles for a Quantum Internet. RFC 9340, March 2023. [doi:10.17487/RFC9340](#).
- [18] Diego Kreutz, Fernando M. V. Ramos, Paulo Verissimo, Christian Esteve Rothenberg, Siamak Azodolmolky, and Steve Uhlig. Software-defined networking: A comprehensive survey, 2014. [arXiv:1406.0440 \[cs.NI\]](#).

- [19] Charles E. Leiserson, Neil C. Thompson, Joel S. Emer, Bradley C. Kuszmaul, Butler W. Lampson, Daniel Sanchez, and Tao B. Schardl. There’s plenty of room at the top: What will drive computer performance after moore’s law? *Science*, 368(6495):eaam9744, 2020. [doi:10.1126/science.aam9744](https://doi.org/10.1126/science.aam9744).
- [20] SujayKumar Reddy M and Chandra Mohan B. Comprehensive analysis of bb84, a quantum key distribution protocol, 2023. [arXiv:2312.05609 \[quant-ph\]](https://arxiv.org/abs/2312.05609).
- [21] Ruben B. Mendez, Juan P. Brito, Rafael J. Vicente, A. Aguado, Antonio Pastor, Diego Lopez, V. Martin, and Victor Lopez. Quantum abstraction interface: Facilitating integration of qkd devices in sdn networks. In *2020 22nd International Conference on Transparent Optical Networks (ICTON)*, pages 1–4, 2020. [doi:10.1109/ICTON51198.2020.9203073](https://doi.org/10.1109/ICTON51198.2020.9203073).
- [22] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, Cambridge, UK, 10th anniversary edition edition, 2010.
- [23] OpenConfig Working Group. Openconfig. <https://www.openconfig.net>, 2026. Accessed: 2026-07-28.
- [24] PicoQuant GmbH. *Multichannel Time-Correlated Single Photon Counting Systems and Time Taggers with USB Interface User Manual*. v4.0.0.0. https://downloads.picoquant.com/manuals/MultiHarp150_160_Manual.pdf.
- [25] Daisuke Sakuma, Tomoki Tsuno, Hikaru Shimizu, Yuki Kurosawa, Monet Tokuyama Friedrich, Kentaro Teramoto, Amin Taherkhani, Andrew Todd, Yosuke Ueno, Michal Hajdušek, Rikizo Ikuta, Rodney Van Meter, Toshihiko Sasaki, and Shota Nagayama. Q-fly: An optical interconnect for modular quantum computers, 2025. [arXiv:2412.09299 \[quant-ph\]](https://arxiv.org/abs/2412.09299).
- [26] SCONTEL. *SCONTEL OPRS-CCR/MCU. USER MANUAL*, 2021. v3.3. https://www.scontel.ru/wp-content/uploads/2021/03/User_Manual__SCONTEL_OPRS_CCR_MCU.pdf.
- [27] Kento Samuel Soon, Michal Hajdušek, Shota Nagayama, Naphan Benchasattabuse, Kentaro Teramoto, Ryosuke Satoh, and Rodney Van Meter. An implementation and analysis of a practical quantum link architecture utilizing entangled photon sources. In *2024 International Conference on Quantum Communications, Networking, and Computing (QCNC)*, pages 25–32. IEEE, 2024. [doi:10.1109/QCNC62729.2024.00014](https://doi.org/10.1109/QCNC62729.2024.00014).

- [28] Amin Taherkhani, Andrew Todd, Lars Matsumiya, and Yifeng Shen. PnPQ. [doi:10.5281/zenodo.16886214](https://doi.org/10.5281/zenodo.16886214).
- [29] Amin Taherkhani, Andrew Todd, Kentaro Teramoto, Rodney Van Meter, and Shota Nagayama. Automatic configuration protocols for optical quantum networks, 2025. [arXiv:2504.19613 \[quant-ph\]](https://arxiv.org/abs/2504.19613).
- [30] Aman Yacob Tekleab, Yifeng Shen, Yoshii Yutaro, Amin Taherkhani, Rodney Van Meter, and Shota Nagayama. A distributed switching protocol for quantum networks, 2026. QCE 2026, to appear. [arXiv:2605.07295 \[quant-ph\]](https://arxiv.org/abs/2605.07295).
- [31] Thorlabs, Inc. *MEMS Fiber Optic Switch Kits OSW12(22)-xxxE Operation Manual*, 2018. <https://media.thorlabs.com/globalassets/items/o/os/osw/osw12-488e/16524-d02.pdf?v=0116014543>.
- [32] Thorlabs, Inc. *ODL100(/M) Optical Delay Line User Guide*, 2021. <https://media.thorlabs.com/globalassets/items/o/od/odl/odl100/ttn126351-d02.pdf?v=0116014409>.
- [33] Thorlabs, Inc. *K10CR2(/M) Precision Motorized Rotation Stage User Guide*, 2024. https://media.thorlabs.com/globalassets/items/k/k1/k10/k10cr2_m/etn064249-d02.pdf?v=0116115646.
- [34] Thorlabs, Inc. *MPC220 and MPC320 Motorized Fiber Polarization Controller User Guide*, 2024. <https://media.thorlabs.com/globalassets/items/m/mp/mpc/mpc320/etn047599-d02.pdf?v=0116012425>.
- [35] Thorlabs, Inc. *Thorlabs Motion Controllers Host-Controller Communications Protocol*, 2024. Issue 39. Vendor documentation archived by the author.
- [36] Thorlabs, Inc. *ODL100(/M) Optical Delay Line User Guide*, 2026. <https://media.thorlabs.com/globalassets/items/k/kb/kbd/kbd101/etn017656-d03.pdf?v=0512035221>.
- [37] Andrew Todd and Kentaro Teramoto. tdc_toolkit. [doi:10.5281/zenodo.16932740](https://doi.org/10.5281/zenodo.16932740).
- [38] Rodney Van Meter. *Quantum Networking*. John Wiley & Sons, 2014. [doi:10.1002/9781118648919](https://doi.org/10.1002/9781118648919).

- [39] Rodney Van Meter, Naphan Benchasattabuse, and Amin Taherkhani. A Quantum Network Architecture. Internet-Draft draft-van-meter-qirg-quantum-network-architecture-00, 2026. <https://www.ietf.org/archive/id/draft-van-meter-qirg-quantum-network-architecture-00.html>.
- [40] Rodney Van Meter, Monet Tokuyama, Andrew Todd, Amin Taherkhani, and Shota Nagayama. Network node specifications, 2026. Private repository; access restricted to members of the Moonshot Nagayama Project. <https://github.com/moonshot-nagayama-pj/network-node-specifications>.
- [41] Stephanie Wehner, David Elkouss, and Ronald Hanson. Quantum internet: A vision for the road ahead. *Science*, 362(6412):eaam9288, 2018. [doi:10.1126/science.aam9288](https://doi.org/10.1126/science.aam9288).
- [42] William K. Wootters and Wojciech H. Zurek. A single quantum cannot be cloned. *Nature*, 299(5886):802–803, 1982. [doi:10.1038/299802a0](https://doi.org/10.1038/299802a0).
- [43] Se-young Yu, Liang Zhang, Ezra Kissel, Wenji Wu, and Inder Monga. An extensible control plane software architecture for quantum networking research. In *Proceedings of the 2nd Workshop on Quantum Networks and Distributed Quantum Computing*, QuNet '25, page 35–41, New York, NY, USA, 2025. Association for Computing Machinery. [doi:10.1145/3749096.3750033](https://doi.org/10.1145/3749096.3750033).